Cloud Computing

Exercise – 7 Total points: 10

Assuming most of you are using Linux machines. If you are using Windows machine, install Oracle VirtualBox and have an Ubuntu virtual machine. https://www.osboxes.org/virtualbox-images/

Follow the instruction at https://docs.docker.com/engine/install/ubuntu/ for installing Docker.

To run Docker commands with non-root privileges

- Create a docker group (If it's already created then ignore): `sudo groupadd docker`
- Add a user to docker group: `sudo usermod -aG docker $USER`
- Activate the changes: `newgrp docker`

1. Practicing Docker commands

You can have a look into basic docker commands here: https://docs.docker.com/engine/reference/commandline/docker/

- Docker hub is a hosted repository service provided by Docker for finding and sharing container images with your team. (https://www.docker.com/products/docker-hub)
- Create a login account at https://hub.docker.com/signup
- Login into your Docker account from Docker host terminal: `docker login`
    - Provide input to the following:
        - Username: your docker hub id
        - Password: Docker hub password
- Pull an image from docker hub: `docker pull ubuntu`
- Check the downloaded images in local repository: `docker images`
- Run a simple ubuntu container, detachedmode (-d), -it runs interactively (so you get a pseudo-TTY with STDIN) : `docker run -dit -p 80:80 --name <<yourname>> ubuntu`
    - <<yourname>> = please type your name
- Get the bash shell of container with non-detached mode: `docker exec -it <container_name> sh`
    - Here you can start configuring your container: install packages, modify files, etc. Alternatively, you can also run commands "outside" of the container using *docker exec*, as we will do in next steps.
    - Exit from the container: `exit`
- Connect to container and update the apt repo: `docker exec -dit <container_name> apt-get update`
- Install http server: `docker exec -it <container_name> apt-get install apache2`
- Check the status of http server: `docker exec -it <container_name> service apache2 status`
- If not running, start the http server: `docker exec -it <container_name> service apache2 start`
- Check the webserver running container host machine : `curl localhost:80`
- Check the ip address of the container: `docker inspect <container_id> | grep -i "IPAddress"`

- Commit the Docker container changes into Docker image `docker commit -m "added apache2 web server" -a "your_name" <container_id> "image_name"`
- Check the newly created image using `docker images`

- Now, stop and delete your container

- **Host directory as a data volume:** Here you are mounting a host directory in a container and this is useful for testing the applications. For example, you store source code in the host directory and mount in the container, the code changed in host directory file can affect the application running in the container.
  - Accessing a host file system on container with read only and read/write modes:
    - Create directory with name test: `mkdir test && cd test`, Create a file: `touch <<yourname>>.txt`
    - Run a container and -v parameter to mount the host directory to the container
      - Read only: `docker run -dit -v /home/osboxes/satish/DDPC/ex_1/:/home/:ro --name vol1 ubuntu`
      - Access the file in a container in the path /home and try to create a new file (you should see access denied) from container `docker exec -it vol1 sh`, `cd /home`, `ls`, `exit`.
      - Read/write: `docker run -dit -v /home/osboxes/satish/DDPC/ex_1/:/home/:rw --name vol2 ubuntu`, `docker exec -it vol2 sh`, `cd /home`, `ls`,
      - Try to create some text files, `exit`.
      - You can see the created files in host machine: `cd /home/osboxes/satish/DDPC/ex_1`, `ls`
  - NB! Take the screenshot here (1 point) `docker ps`
    - Stop and delete the containers : `docker stop vol1`, `docker rm vol1`

- **Data volume containers:** A popular practice with Docker data sharing is to create a dedicated container that holds all of your persistent shareable data resources, mounting the data inside of it into other containers once created and setup.
  - Create a data volume container and share data between containers.
    - Create a data volume container `docker run -dit -v /data --name data-volume ubuntu`, `docker exec -it data-volume sh`
    - Go to volume and create some files: `cd /data && touch file1.txt && touch file2.txt`
    - Exit the container `exit`
    - Run another container and mount the volume as earlier container: `docker run -dit --volumes-from data-volume --name data-shared ubuntu`, `docker exec -it data-shared sh`
    - Go to the data directory in the created container and list the files: `cd /data && ls`
  - NB! Take the screenshot here (1 point) `docker ps`

2. Configuring web application & installing software

We will now set up a simplistic Python web application on the local machine, setting up the necessary software and dependencies. The application is built with the Flask micro web framework. The app represents a messaging board that stores posted messages into a json-structured text-file.

1. Install Python Flask webserver on the instance
    - First, fetch the source code of the application with git, storing it into the folder "lab1app":
        - `git clone` https://github.com/cloud-and-smart-labs/DDPC-2024-ex1.git `lab1app`
        - Enter the application src directory: `cd lab1app`
    - This app uses the *pip* package Flask as a dependency. We will create a Python *venv* (virtual environment) for this application and install the packages for the app within the *venv*.
        - Update the package lists about available software and install the *python3-venv* package:
            - `sudo apt update && sudo apt install python3.8-venv`
        - Create a *venv* named "env", and activate it:
            - `python3 -m venv env`
            - `source env/bin/activate`
            - After activating, you should see the "(env)" appear in front of your username in the terminal session.
            - Note: You can use `deactivate` to exit out of the venv
        - Use pip to install the necessary dependencies of this application (defined in requirements.txt):
            - `pip install -r requirements.txt`
    - Start the Flask web application
        - `flask run --host=0.0.0.0`
            - This will start the Flask server on port 5000.
            - Note: you need to be in the *lab1app* folder!
            - We set the "host=0.0.0.0" parameter to ensure the webserver listens to external traffic too, not only local traffic.
        - Or, to set it to run in the background, if you wish :)
            - `nohup flask run --host=0.0.0.0 &`
            - *nohup* ensures the program stays running after we log out
            - Note: if you want to kill python program running in the background, you can use `fuser -n tcp -k 5000` - this kills the process listening on port 5000
2. Check that the installation of the Flask web server is successful
    - Try accessing the address of your virtual machine on port 5000 from a browser.
        - E.g. http://172.17.67.124:5000
        - This, in theory, should display the `lab1app/templates/home.html` page that's being provided by your web server.
    - Let's use an alternative approach to verify our web server is running
        - Use the `wget` command to download the webpage from your server.
        - On the instance command line run `wget localhost:5000`.
            - This should download index.html file into the currently active directory, which among other HTML codes should contain a string "Message board".

    - Use `less index.html` command to check the downloaded file content from the command line
    - Alternatively, you can use command line web-browser `sudo apt install lynx` and `lynx localhost:5000`
3. Modify (or replace) the current home.html file at `lab1app/templates/home.html` to change the web page content.
    - How exactly you change its content is up to you, but it should at least contain your **Full Name**, so it is possible to visually see that you have modified it.
    - To deploy the changes, you need to stop and re-start the Flask server!
    - Command line file editor `nano` can be used to modify file contents.
    - Feel free to replace the whole HTML file with a new one.
    - NB! Take the screenshot here (2 points)

3. Creating Docker Image using Dockerfile

The task is to create your own docker image and try to run the same. More information about Dockerfile (https://docs.docker.com/engine/reference/builder/). A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image. Using docker build users can create an automated build that executes several command-line instructions in succession.

Some of Dockerfile commands:

FROM	Set the base image
LABEL	Add a metadata to image
RUN	Execute the command and execute the commands
CMD	Allowed only once
EXPOSE	Container listen on the specific ports at runtime
ENV	Set environment variables
COPY	Copy the files or directories into container's file system
WORKDIR	Set working directory

The scenario is to create a docker image and deploy the flask application from Part 2 using the Docker container. In this exercise, we are going to perform two tasks:

1. Create docker **Image** for flask application (Message Board)
2. **Deploy** flask application container using the host directory as a volume (bind volumes) to store messages in the host directory (data.json in flask application).

3.1: Create docker image for flask application

- Let us write a Dockerfile to create a docker image as shown below:

- Create a Dockerfile (inside the lab1app directory) `sudo nano Dockerfile`
- Add the following set docker commands:
  FROM command chooses the base image for the container, here it is python:slim-buster
  COPY command will copy the code from host directory to container directory lab1app (You are copying the Messageboard code to container image)
  WORKDIR Set the current working directory when your container starts.
  RUN command will run the specified commands on the shell.
  CMD command will run the flask application at a specified host address and port.

```
FROM python:slim-buster

WORKDIR /lab1app

COPY . .

RUN pip3 install -r requirements.txt

ENV FLASK_APP=app.py

ENV FLASK_RUN_HOST=0.0.0.0

EXPOSE 5000

CMD [ "flask", "run"]
```

- Note: Clean your folder off the python env trials of part 2.
- Build the container image using the following command **docker build** as `docker build -t <flask_task1_lastname> .` **inside the lab1app** folder.
  - Now, look at the output of the build command where a set of docker commands are executed in the order you have written in Dockerfile.
  - After a successful build, you should see your built image in the list of local images when calling `docker images`
- Now, run the container with the following options:
  - port mapping : `host(80) to container(5000)`
  - image name: `<flask_task1_lastname>`
  - Name of container: `<task1_your_lastname>`
- By now, you should see the application running at `http://VM_IP:80`
- Stop and delete the container
- NB! Take the screenshots here of both the terminal with commands and the browser (2 points)

3.2: Deploy Flask app container with a mounted-volume based storage

Deploy flask application container using the host directory as a volume (bind volumes) to store messages in the host directory (data.json in flask application).

- Copy the data.json file (inside the lab1app folder) into the ubuntu user home folder inside the VM in the path (e.g. `/home/ubuntu/data.json`).
- Now run the container with the following options:
  - detached mode (-dit)
  - Container image `<flask_task1_lastname>`
  - Name of container `<task2_your_lastname>`
  - port mapping host(80), container(5000)

    - Volume (-v) `/home/ubuntu/data.json:/lab1app/data.json`
        - This will mount the file you previously created (inside the host file) into the correct location inside the container.
- By now, you should see the application running at `http://VM_IP:80` and add few messages
- Check the content of data.json `cat /home/ubuntu/data.json`, here you can see the added messages.
- Stop and remove the container
- Run the container again and should see the data of previous container's application in http://VM_IP:80 web page
- NB! Take the screenshot of `cat /home/ubuntu/data.json` inside the VM and `docker exec -it <task2_your_lastname> cat /lab1app/data.json` (2 points)
- Stop and delete the container

Exercise 3.3. Shipping a Docker image to Docker hub

- **Push the docker image to docker hub**
    - Initially login into your docker account from docker host terminal: `docker login`
        - Provide input to the following:
            - Username: your docker hub id
            - Password: Docker hub password
    - Tag your image before pushing into docker hub: `docker tag <flask_task1_lastname> your-dockerhub-id/flask1.0`
    - Finally push the image to docker hub: `docker push your-dockerhub-id/flask1.0`
- **Scenario 1:** Modify the container by creating some directories and save the changes in the image using the docker commit command. (1 point)
- **Scenario 2:** Export the container as a tar file by using the command docker export. (1 point)
    - NB! Take the screenshot of the docker hub account showing your image and other screenshots showing the work done

Follow these guidelines while submitting the exercises

1. Put a pdf file with your answers and screenshots for the questions and name it as your studentid_QE7
2. The screenshots are meant to show that the work is done and by you. So, have enough proofs in your answer sheet.
3. **Note**: Do not blindly copy the commands. Try to understand them. The provided commands are to guide you with the initial process.
4. Do not copy code from internet. If you copy, please leave a comment with the URL on that code/answer sheet
5. Plagiarism will not be tolerated and dealt through the formal channels of the university